

Reviewer Pack — Curto-Fialkow Hankel Defect Separates Read-Twice IP₂ BPs

Entry 9f09553bfd37 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 04:32:37 UTC

Curto-Fialkow Hankel Defect Separates Read-Twice IP₂ BPs

Entry ID: 9f09553bfd37 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 04:32:37 UTC

1. Conjecture

Field A (mathematical branch): Curto-Fialkow truncated moment problem theory (Hankel positivity and flat-extension rank, the real-algebraic-geometry foundation of the Lasserre SoS hierarchy; Curto-Fialkow 1996, Lasserre 2001) applied to layer-conditional empirical moments of branching programs **Field B** (complexity object): Read-twice oblivious branching program size for IP₂ on $2n$ variables in adversarial order $x_1, \dots, x_n, y_1, \dots, y_n$ (Borodin-Razborov-Smolensky 1993 open separation)

Statement:

Let P be a read-twice oblivious BP with state set Q ($|Q|=s$) computing a Boolean function on $N=2n$ variables, and let $q(z, \square)$ denote its state on input z after $\square$ steps. For each state $q \in Q$ and each layer $\square \in \{1, \dots, 2N\}$, define the centered conditional moment sequence $m^{\{q, \square\}}(P) = 2^{\{-N\}} \sum_z \mathbb{I}[q(z, \square)=q] \cdot (IP_2(z)-1/2)^k$ for $k=0, \dots, \lfloor N/4 \rfloor$, and let $H^{\{q, \square\}}(P)$ be the $(d+1) \times (d+1)$ Hankel matrix with $d=\lfloor N/8 \rfloor$ and entries $m^{\{q, \square\}}(P)$. Define the positivity defect $\delta(P) = \max_{\{q, \square\}} \#\{\text{negative eigenvalues of } H^{\{q, \square\}}(P)\}$. Conjecture: (i) for every read-twice oblivious BP P of size s , $\delta(P) \leq 8 \cdot \lceil \log_2(s+1) \rceil$; (ii) every read-twice oblivious BP P that exactly computes IP₂ in the adversarial schedule satisfies $\delta(P) \geq \lceil \sqrt{N/2} \rceil$.

Rationale (proposer's reasoning):

Curto-Fialkow theory equates degree- $2d$ SoS feasibility with the existence of flat positive extensions of truncated Hankel sequences, so a small BP forces low-rank flat extensions of its layer-conditional empirical moments — a structural constraint never previously exploited for BP lower bounds. IP₂'s bilinear $\oplus$ -structure should force every conditional sub-population reaching a given state at a given layer to retain non-trivial bilinear correlation between the unread x -bits and the moments, obstructing flat positive Hankel extension and producing many negative eigenvalues. The conjectured gap $\log(s)$ vs $\sqrt{N}$ would yield $s \geq 2^{\{\Omega(\sqrt{N})\}}$ for read-twice IP₂, attacking the BRS-1993 problem via the SoS-moment side rather than via communication or partition arguments.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 74c0bc0f71ba9796

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{3, 4, 5, 6\}$, widths $w \in \{2, 3, 4, 6, 8, 12\}$, and 5 seeds (30 random read-twice BP-s/seed) plus the explicit IP_2 BP for $n \in \{3, 4, 5\}$: every random BP must satisfy $\delta(P) \leq 8 \cdot \lceil \log_2(s+1) \rceil$ AND every IP_2 BP must satisfy $\delta(P) \geq \lceil \sqrt{N/2} \rceil$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - read-twice oblivious branching program inner product IP2 lower bound - Hankel moment matrix flat extension branching program complexity - Lasserre sum of squares hierarchy communication complexity inner product

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_rank(A):
```

```

n = len(A)
m = len(A[0])
rank = 0
for i in range(min(n, m)):
    if any(A[i][j] != 0 for j in range(m)):
        rank += 1
        for k in range(i + 1, n):
            factor = Fraction(A[k][i], A[i][i])
            for j in range(i, m):
                A[k][j] -= factor * A[i][j]
return rank

def count_negative_eigenvalues(H):
    n = len(H)
    count = 0
    for i in range(n):
        if H[i][i] < 0:
            count += 1
    return count

def generate_random_bp(n, w, seed):
    random.seed(seed)
    Q = list(range(w))
    N = 2 * n
    P = {}
    for q in Q:
        for l in range(2 * N):
            P[(q, l)] = [0] * (N + 1)
    for z in itertools.product([0, 1], repeat=N):
        q = random.choice(Q)
        for l in range(2 * N):
            k = sum((z[i] - 0.5) ** 2 for i in range(N))
            P[(q, l)][k] += 1
    return P, Q, N

def generate_ip2_bp(n):
    N = 2 * n
    Q = list(itertools.product([0, 1], repeat=n))
    P = {}
    for q in Q:
        for l in range(2 * N):
            P[(q, l)] = [0] * (N + 1)
    for z in itertools.product([0, 1], repeat=N):
        q = tuple(z[:n])
        for l in range(2 * N):
            k = sum((z[i] - 0.5) ** 2 for i in range(N))
            P[(q, l)][k] += 1
    return P, Q, N

def compute_delta(P, Q, N):
    d = N // 8
    delta = 0
    for q in Q:
        for l in range(2 * N):
            m = P[(q, l)]
            H = [[m[i + j] for j in range(d + 1)] for i in range(d + 1)]

```

```

        rank = matrix_rank(H)
        if rank < d + 1:
            delta += 1
    return delta

def run_trial(seed):
    random.seed(seed)
    n_values = [3, 4, 5, 6]
    w_values = [2, 3, 4, 6, 8, 12]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for w in w_values:
            P, Q, N = generate_random_bp(n, w, seed)
            delta = compute_delta(P, Q, N)
            s = len(Q)
            bound = 8 * math.ceil(math.log2(s + 1))
            if delta > bound:
                conjecture_holds = False
                counterexample = f"Random BP with n={n}, w={w}, delta={delta} exceeds bound {bound}"
                break
            metric_values.append(delta)
            instances_tested += 1

    if not conjecture_holds:
        break

    if conjecture_holds:
        for n in [3, 4, 5]:
            P, Q, N = generate_ip2_bp(n)
            delta = compute_delta(P, Q, N)
            bound = math.ceil(math.sqrt(N) / 2)
            if delta < bound:
                conjecture_holds = False
                counterexample = f"IP2 BP with n={n}, delta={delta} falls below bound {bound}"
                break
            metric_values.append(delta)
            instances_tested += 1

    if not metric_values:
        return {
            "metric_name": "delta",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances tested"
        }

    metric_value = sum(metric_values) / len(metric_values)
    return {
        "metric_name": "delta",
        "metric_value": metric_value,
        "instances_tested": instances_tested,

```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = sys.argv[1:]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]

    results = []
    for seed in seeds:
        result = run_trial(int(seed))
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_metric_values")
        sys.exit(0)

    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8925cd5b.py", line 155, in <module>
    result = run_trial(int(seed))
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8925cd5b.py", line 104, in run_trial
    P, Q, N = generate_random_bp(n, w, seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8925cd5b.py", line 64, in generate_random_bp
    P[(q, l)][k] += 1
    ~~~~~^
TypeError: list indices must be integers or slices, not float

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `generate_random_bp` before producing any data, so neither the support nor falsification condition could be evaluated. | next: Fix the indexing bug in `generate_random_bp` (cast the index `k` to `int`) and rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	138270
2	preregistration	claude_max	opus	0	0	6425
3	novelty	claude_max	opus	0	0	4139
4	novelty	claude_max	opus	0	0	11481
5	test_gen	mistral	codestral-latest	0	0	312224
6	test_gen	mistral	codestral-latest	0	0	315083
7	test_gen	mistral	codestral-latest	0	0	321249
8	test_gen	mistral	codestral-latest	0	0	310485
9	judge	claude_max	opus	0	0	3609

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1422965 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/9f09553bfd37.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9f09553bfd37.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9f09553bfd37.tar.gz` (if generated)